- I accessed both of these Ais through their websites in google chrome https://claude.ai/ and https://chatgpt.com/

Prompt:

generate C++ code using objects, not functions, to implement
the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
• The program will prioritize emails for a busy company CEO.
• You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
• You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
• You must create objects from scratch. Do not include pre-existing heap modules.
• Test File format:
Command Description
EMAIL <sender category>,
<subject line>,
<date>
The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last
If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.
EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY
NEXT
Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:

Date:
READ CEO has read email and dealt with it
COUNT Display current count of unread emails
• The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).
• Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

• Your program should display this:

```
There are 6 emails to read.
Next email:
Sender: Boss
Subject: Never Mind
Date: 01-03-2025
Next email:
Sender: Boss
Subject: Important
Date: 12-20-2024
There are 4 emails to read.
```

Adequate Prologue Comments:
• Name of program contained in the file (e.g., EECS 348 Assignment 1)
• Brief description of the program, e.g.,
o C program that picks a secret number between 1 and 10.
o The user gets up to 3 tries to guess the number.
• Inputs e.g., none, for a function, it would be the parameters passed to it
• Output, e.g., terminal output between user and program
• All collaborators

• Other sources for the code ChatGPT, stackOverflow, etc.
• Author's full name
• Creation date: The date you first create the file, i.e., the date you write this comment
• Revision date: The date you revised the file
• Revisions: e.g, convert C code to C++

Overview:

Claude.cpp is the better program and is the program I chose to submit for this for my baseline program. It

Criteria	Chat.cpp	Claude.cpp	Better?
Correctness	Has correct input/output; miss phrases subjects containing commas; does not define behavior on bad dates inputs	Has correct input; can handle commas in subjects and bad dates correctly	claude .pp
Execution time	O(log n) per EMAIL and READ, O(1) per NEXT and COUNT	Same complexity	Tie
Space complexity	O(n)	Same complexity	Tie
Maintainability	723 lines, lots of vertical spacing, comments are restating code not explaining it	345 lines, small helper functions, comments describe inputs and outputs	Claude

Correctness

Case	Chat.cpp	Claude.cpp
READ on an empty inbox	Prints nothing	Prints “No emails to read”
Unknown command or EMAIL with fewer than two commas	Ignored	Prints Error message
No filename argument	Reads commands from standard input	Prompts for a filename
Subject containing a comma (Hello, are you free?)	Subject containing a comma (Hello, are you free?) Splits on the first two commas: subject becomes Hello and the date is garbage	Splits on the first and last comma: subject and date are both correct

The last row is a defect in chat.cpp which is why Claude has the edge in correctness.

Execution time

Both programs have the same complexity.

Maintainability

Cluade.cpp is easier to maintain, it has the same design but is 345 lines of code versus chat.cpp 723. Claude also has smaller functions and more descriptive comments.

Changes

The one change I made was: Move instead of copy in swapItems (lines 183 to 187). Replace the body with std::swap(items[first], items[second]); and add #include <utility>. This removes three string copies per swap.

Original Code

```
/*
 * Program: EECS 348 Assignment 2 - CEO Email Priority Queue
 *
 * Description:
 *   C++ program that prioritizes a CEO's unread emails using a MaxHeap.
 *   Emails are prioritized first by sender category and then by date.
 *   Newer emails from the same sender category are given higher priority.
 *
 * Input:
 *   Commands read from a test file or standard input:
 *
 *     EMAIL <sender category>,<subject line>,<date>
 *     NEXT
 *     READ
 *     COUNT
 *
 * Output:
 *   Displays the number of unread emails and the highest-priority email
 *   when requested.
 *
 * Collaborators: None
 * Other Sources: ChatGPT (OpenAI)
 * Author: Joseph Akins
 * Creation Date: October 1, 2026
 * Revision Date: October 1, 2026
 *
 * Revisions:
 *   Initial C++ object-oriented implementation using a custom MaxHeap.
 */

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <sstream>

using namespace std;
```

```
/*
 * Email object
 *
 * Stores all information associated with one email.
 */
class Email {
private:
    string sender;
    string subject;
    string date;

    int senderPriority;
    int dateValue;

    long long arrivalOrder;

public:

    /*
     * Constructor
     *
     * Creates an Email object and determines its sender priority
     * and numeric date value.
     */
    Email(const string& senderCategory,
          const string& subjectLine,
          const string& emailDate,
          long long order)
    {
        sender = senderCategory;
        subject = subjectLine;
        date = emailDate;
        arrivalOrder = order;

        /*
         * Higher numbers mean higher priority.
         */
        if (sender == "Boss") {
            senderPriority = 5;
```

```
        }
        else if (sender == "Subordinate") {
            senderPriority = 4;
        }
        else if (sender == "Peer") {
            senderPriority = 3;
        }
        else if (sender == "ImportantPerson") {
            senderPriority = 2;
        }
        else {
            senderPriority = 1;
        }

        /*
         * Convert:
         *
         * MM-DD-YYYY
         *
         * into:
         *
         * YYYYMMDD
         *
         * Example:
         *
         * 01-03-2025
         *
         * becomes:
         *
         * 20250103
         *
         * This makes dates easy to compare.
         */
        int month;
        int day;
        int year;

        char dash1;
        char dash2;
```

```
        stringstream dateStream(date);

        dateStream >> month
                   >> dash1
                   >> day
                   >> dash2
                   >> year;

        dateValue = year * 10000
                  + month * 100
                  + day;
    }



    /*
     * Determines whether this email has higher priority
     * than another email.
     */
    bool hasHigherPriorityThan(const Email& other) const {

        /*
         * First compare sender category.
         */
        if (senderPriority != other.senderPriority) {

            return senderPriority > other.senderPriority;
        }


        /*
         * If they have the same sender category,
         * the newer email gets priority.
         */
        if (dateValue != other.dateValue) {

            return dateValue > other.dateValue;
        }


        /*
```

```
         * If category AND date are identical,
         * keep the email that entered first ahead.
         *
         * This simply gives predictable behavior
         * when two emails are otherwise identical.
         */
        return arrivalOrder < other.arrivalOrder;
    }


    /*
     * Displays the email in the required format.
     */
    void display() const {

        cout << "Sender: " << sender << endl;
        cout << "Subject: " << subject << endl;
        cout << "Date: " << date << endl;
    }
};



/*
 * MaxHeap object
 *
 * Implements a MaxHeap using a list/array-based representation.
 *
 * No built-in C++ heap functions are used.
 */
class MaxHeap {
private:

    /*
     * Vector acts as the list used to store the heap.
     *
     * For index i:
     *
     * Parent      = (i - 1) / 2
     * Left child  = 2i + 1
```

```
     * Right child = 2i + 2
     */
    vector<Email> heap;


    /*
     * Swaps two Email objects inside the heap.
     */
    void swapEmails(int first, int second) {

        Email temp = heap[first];

        heap[first] = heap[second];

        heap[second] = temp;
    }


    /*
     * Moves a newly inserted email upward until
     * the MaxHeap property is restored.
     */
    void bubbleUp(int index) {

        while (index > 0) {

            int parent = (index - 1) / 2;


            /*
             * If the child has greater priority than
             * the parent, swap them.
             */
            if (heap[index].hasHigherPriorityThan(heap[parent])) {

                swapEmails(index, parent);

                index = parent;
            }
            else {
```

```
            break;
        }
    }
}


/*
 * Moves an email downward after the root is removed.
 */
void bubbleDown(int index) {

    int heapSize = static_cast<int>(heap.size());


    while (true) {

        int leftChild = 2 * index + 1;

        int rightChild = 2 * index + 2;

        int highest = index;


        /*
         * Check the left child.
         */
        if (leftChild < heapSize &&
            heap[leftChild].hasHigherPriorityThan(heap[highest])) {

            highest = leftChild;
        }


        /*
         * Check the right child.
         */
        if (rightChild < heapSize &&
            heap[rightChild].hasHigherPriorityThan(heap[highest])) {
```

```
                highest = rightChild;
            }


            /*
             * If index is still the highest priority,
             * the heap is correct.
             */
            if (highest == index) {

                break;
            }


            swapEmails(index, highest);

            index = highest;
        }
    }


public:

    /*
     * Inserts an Email into the MaxHeap.
     */
    void insert(const Email& email) {

        /*
         * Put new email at end of list.
         */
        heap.push_back(email);


        /*
         * Restore heap property.
         */
        bubbleUp(static_cast<int>(heap.size()) - 1);
    }
```

```
/*
 * Returns the highest-priority email
 * without removing it.
 */
const Email* peek() const {

    if (heap.empty()) {

        return nullptr;
    }


    /*
     * Highest-priority item in a MaxHeap
     * is always index 0.
     */
    return &heap[0];
}


/*
 * Removes the highest-priority email.
 */
void removeMax() {

    /*
     * Nothing to remove.
     */
    if (heap.empty()) {

        return;
    }


    /*
     * If only one email exists, simply remove it.
     */
    if (heap.size() == 1) {
```

```
            heap.pop_back();

            return;
        }


        /*
         * Move last email into the root.
         */
        heap[0] = heap.back();


        /*
         * Remove last position.
         */
        heap.pop_back();


        /*
         * Restore MaxHeap property.
         */
        bubbleDown(0);
    }


    /*
     * Returns number of emails currently in heap.
     */
    int count() const {

        return static_cast<int>(heap.size());
    }


    /*
     * Determines whether the heap is empty.
     */
    bool empty() const {

        return heap.empty();
```

```
    }
};



/*
 * CEOInbox object
 *
 * Handles commands from the input file and uses
 * a MaxHeap object to manage the CEO's emails.
 */
class CEOInbox {
private:

    MaxHeap inbox;

    long long nextArrivalOrder;


    /*
     * Removes spaces, tabs, and line endings
     * from the beginning and end of a string.
     */
    string trim(const string& text) const {

        string whitespace = " \t\r\n";


        size_t start =
            text.find_first_not_of(whitespace);


        /*
         * String contains only whitespace.
         */
        if (start == string::npos) {

            return "";
        }
```

```
    size_t end =
        text.find_last_not_of(whitespace);


    return text.substr(
        start,
        end - start + 1
    );
}


/*
 * Handles an EMAIL command.
 *
 * Example:
 *
 * EMAIL Boss,Never Mind,01-03-2025
 */
void addEmail(const string& line) {

    /*
     * Remove "EMAIL " from the beginning.
     */
    string emailData = line.substr(6);


    /*
     * Find the two commas.
     */
    size_t firstComma =
        emailData.find(',');

    size_t secondComma =
        emailData.find(',', firstComma + 1);


    /*
     * Invalid input protection.
     */
```

```
        if (firstComma == string::npos ||
            secondComma == string::npos) {

            return;
        }


        /*
         * Extract sender.
         */
        string sender = trim(
            emailData.substr(
                0,
                firstComma
            )
        );


        /*
         * Extract subject.
         *
         * Subjects may contain spaces,
         * so we cannot simply use >>.
         */
        string subject = trim(
            emailData.substr(
                firstComma + 1,
                secondComma - firstComma - 1
            )
        );


        /*
         * Extract date.
         */
        string date = trim(
            emailData.substr(secondComma + 1)
        );
```

```
        /*
         * Create Email object.
         */
        Email newEmail(
            sender,
            subject,
            date,
            nextArrivalOrder
        );


        nextArrivalOrder++;


        /*
         * Insert Email object into heap.
         */
        inbox.insert(newEmail);
    }


    /*
     * Handles NEXT command.
     *
     * NEXT only displays the highest-priority email.
     *
     * It DOES NOT remove it.
     */
    void showNext() const {

        const Email* nextEmail =
            inbox.peek();


        /*
         * Handle NEXT with no emails.
         */
        if (nextEmail == nullptr) {

            cout << "No emails to read." << endl;
```

```
            return;
        }


        cout << "Next email:" << endl;


        nextEmail->display();
    }


    /*
     * Handles READ command.
     *
     * READ removes the highest-priority email.
     *
     * It does not matter whether NEXT was called first.
     */
    void readNext() {

        inbox.removeMax();
    }


    /*
     * Handles COUNT command.
     */
    void showCount() const {

        cout << "There are "
             << inbox.count()
             << " emails to read."
             << endl;
    }


public:

    /*
```

```
 * Constructor
 */
CEOInbox() {

    nextArrivalOrder = 0;
}


/*
 * Reads and executes every command from
 * the input stream.
 */
void processStream(istream& input) {

    string line;


    while (getline(input, line)) {

        /*
         * Remove extra whitespace or Windows \r.
         */
        line = trim(line);


        if (line.empty()) {

            continue;
        }


        /*
         * EMAIL command
         */
        if (line.rfind("EMAIL ", 0) == 0) {

            addEmail(line);
        }
```

```
            /*
             * NEXT command
             */
            else if (line == "NEXT") {

                showNext();
            }


            /*
             * READ command
             */
            else if (line == "READ") {

                readNext();
            }


            /*
             * COUNT command
             */
            else if (line == "COUNT") {

                showCount();
            }
        }
    }
};



/*
 * Main program
 */
int main(int argc, char* argv[]) {

    /*
     * Create CEO Inbox object.
     */
    CEOInbox ceoInbox;
```

```
/*
 * If a filename is given:
 *
 * ./program test.txt
 *
 * read commands from that file.
 */
if (argc >= 2) {

    ifstream inputFile(argv[1]);


    /*
     * Make sure file opened successfully.
     */
    if (!inputFile.is_open()) {

        cerr << "Unable to open input file."
             << endl;

        return 1;
    }


    /*
     * Process commands.
     */
    ceoInbox.processStream(inputFile);
}


/*
 * This also allows:
 *
 * ./program < test.txt
 */
else {
```

```
        ceoInbox.processStream(cin);
    }


    return 0;
}
```

```
/*
 * Program name: EECS 348 Assignment - CEO Email Priority Queue (GenAI
version)
 *
 * Description:
 *   C++ program that prioritizes the emails in a busy CEO's Inbox.
 *   Emails are stored in a priority queue that is implemented from
scratch
 *   as a list-based (array-based) MaxHeap. Priority is decided first by
the
 *   sender category (Boss > Subordinate > Peer > ImportantPerson >
 *   OtherPerson) and then by date, with the NEWEST email read first.
 *   The program is built from three classes:
 *     Email   - one email and the rule for comparing two emails
 *     MaxHeap - list-based max heap of Email objects
 *     Inbox   - reads the test file and carries out each command
 *
 * Inputs:
 *   A text test file containing one command per line:
 *     EMAIL <sender category>,<subject line>,<date as MM-DD-YYYY>
 *     NEXT
 *     READ
 *     COUNT
 *   The file name may be given as the first command-line argument.
 *   If no argument is given, the program asks the user for the file name.
 *
 * Output:
 *   Terminal output:
 *     NEXT  - displays the sender, subject and date of the highest
 *             priority email (the email stays in the queue)
 *     READ  - removes the highest priority email (no display)
 *     COUNT - displays the number of unread emails
 *
```

```
 * Collaborators: None
 *
 * Other sources for the code: Claude (Anthropic) generative AI
 *
 * Author: Joseph Akins
 *
 * Creation date: 10-01-2026
 *
 * Revision date: 10-01-2026
 *
 * Revisions: Initial version generated with Claude
 */

#include <fstream>
#include <iostream>
#include <string>
#include <vector>

/*
 * Class: Email
 * Represents a single email and knows how to compare itself with another
 * email to decide which one the CEO should read first.
 */
class Email {
public:
    // Default constructor: builds an empty, lowest priority email.
    Email()
        : senderCategory(""), subject(""), date(""),
          categoryRank(0), dateKey(0), arrivalOrder(0) {}

    // Inputs: sender category, subject line, date (MM-DD-YYYY) and the
    //         order in which the email arrived in the Inbox.
    Email(const std::string& sender, const std::string& subjectLine,
          const std::string& dateText, long arrival)
        : senderCategory(sender), subject(subjectLine), date(dateText),
          categoryRank(rankOf(sender)), dateKey(dateKeyOf(dateText)),
          arrivalOrder(arrival) {}

    // Returns true when this email should be read before the other email.
    // Input: the email to compare against.
```

```
    bool hasHigherPriorityThan(const Email& other) const {
        // 1. A more important sender category always wins.
        if (categoryRank != other.categoryRank) {
            return categoryRank > other.categoryRank;
        }
        // 2. Same category: the newest date wins.
        if (dateKey != other.dateKey) {
            return dateKey > other.dateKey;
        }
        // 3. Same category and date: the email that arrived last is
newest.
        return arrivalOrder > other.arrivalOrder;
    }

    // Output: prints the email to the terminal in the required format.
    void display() const {
        std::cout << "Sender: " << senderCategory << std::endl;
        std::cout << "Subject: " << subject << std::endl;
        std::cout << "Date: " << date << std::endl;
    }

private:
    std::string senderCategory; // One of the five sender categories
    std::string subject;        // Subject line (may contain spaces)
    std::string date;           // Date exactly as read: MM-DD-YYYY
    int categoryRank;           // Larger number = read sooner
    int dateKey;                // Date as YYYYMMDD so dates compare as
numbers
    long arrivalOrder;          // Position of the email in the test file

    // Converts a sender category into a numeric rank.
    // Input: sender category string. Output: 5 (Boss) down to 1
(OtherPerson).
    static int rankOf(const std::string& sender) {
        if (sender == "Boss") return 5;
        if (sender == "Subordinate") return 4;
        if (sender == "Peer") return 3;
        if (sender == "ImportantPerson") return 2;
        if (sender == "OtherPerson") return 1;
        return 0; // Unknown category: lowest priority
```

```
    }

    // Converts MM-DD-YYYY into the integer YYYYMMDD.
    // Input: date string. Output: sortable integer (0 if the date is
malformed).
    static int dateKeyOf(const std::string& dateText) {
        if (dateText.length() != 10 || dateText[2] != '-' || dateText[5]
!= '-') {
            return 0;
        }
        int key = 0;
        const int order[8] = {6, 7, 8, 9, 0, 1, 3, 4}; // YYYY, then MM,
then DD
        for (int i = 0; i < 8; i++) {
            char digit = dateText[order[i]];
            if (digit < '0' || digit > '9') {
                return 0;
            }
            key = key * 10 + (digit - '0');
        }
        return key;
    }
};

/*
 * Class: MaxHeap
 * A max heap of Email objects stored in a list (std::vector). The email
 * with the highest priority is always at index 0. For the item at index
i:
 *   parent = (i - 1) / 2, left child = 2i + 1, right child = 2i + 2
 * No pre-existing heap or priority queue modules are used.
 */
class MaxHeap {
public:
    // Adds an email to the heap. Input: the email to add.
    void insert(const Email& email) {
        items.push_back(email);       // Place at the end of the list
        siftUp(size() - 1);           // Move it up to its correct
position
    }
```

```
    // Returns the highest priority email without removing it.
    // The caller must make sure the heap is not empty.
    const Email& peek() const {
        return items[0];
    }

    // Removes the highest priority email.
    // Output: true if an email was removed, false if the heap was empty.
    bool removeMax() {
        if (isEmpty()) {
            return false;
        }
        items[0] = items[size() - 1]; // Move the last item to the root
        items.pop_back();             // Shrink the list
        if (!isEmpty()) {
            siftDown(0);              // Move the new root down into place
        }
        return true;
    }

    // Output: number of emails currently in the heap.
    int size() const {
        return static_cast<int>(items.size());
    }

    // Output: true when the heap holds no emails.
    bool isEmpty() const {
        return items.empty();
    }

private:
    std::vector<Email> items; // The list that stores the heap

    // Index helpers
    int parentOf(int index) const { return (index - 1) / 2; }
    int leftOf(int index) const { return 2 * index + 1; }
    int rightOf(int index) const { return 2 * index + 2; }

    // Exchanges the emails stored at two indexes.
```

```
    void swapItems(int first, int second) {
        Email temp = items[first];
        items[first] = items[second];
        items[second] = temp;
    }

    // Moves the item at index up until its parent has higher priority.
    void siftUp(int index) {
        while (index > 0 &&
               items[index].hasHigherPriorityThan(items[parentOf(index)]))
{
            swapItems(index, parentOf(index));
            index = parentOf(index);
        }
    }

    // Moves the item at index down until both children have lower
priority.
    void siftDown(int index) {
        while (true) {
            int largest = index;
            int left = leftOf(index);
            int right = rightOf(index);

            if (left < size() &&
                items[left].hasHigherPriorityThan(items[largest])) {
                largest = left;
            }
            if (right < size() &&
                items[right].hasHigherPriorityThan(items[largest])) {
                largest = right;
            }
            if (largest == index) {
                return; // Heap property restored
            }
            swapItems(index, largest);
            index = largest;
        }
    }
};
```

```
/*
 * Class: Inbox
 * The CEO's Inbox. Reads the test file and uses a MaxHeap as a priority
 * queue to carry out the EMAIL, NEXT, READ and COUNT commands.
 */
class Inbox {
public:
    Inbox() : arrivalCounter(0) {}

    // Reads the test file and processes every command in it.
    // Input: name of the test file.
    // Output: true if the file was opened, false otherwise.
    bool processFile(const std::string& fileName) {
        std::ifstream file(fileName.c_str());
        if (!file.is_open()) {
            return false;
        }
        std::string line;
        while (std::getline(file, line)) {
            processCommand(trim(line));
        }
        file.close();
        return true;
    }

private:
    MaxHeap queue;       // Priority queue of unread emails
    long arrivalCounter; // Counts emails so ties can be broken by arrival

    // Decides which command a line holds and carries it out.
    // Input: one line of the test file with outer whitespace removed.
    void processCommand(const std::string& line) {
        if (line.empty()) {
            return; // Skip blank lines
        }
        if (line.compare(0, 6, "EMAIL ") == 0) {
            addEmail(line.substr(6));
        } else if (line == "NEXT") {
            showNext();
```

```
        } else if (line == "READ") {
            markRead();
        } else if (line == "COUNT") {
            showCount();
        } else {
            std::cout << "Unknown command: " << line << std::endl;
        }
    }

    // EMAIL command: builds an email and places it in the queue.
    // Input: the text after "EMAIL ", i.e. <category>,<subject>,<date>
    void addEmail(const std::string& details) {
        std::size_t firstComma = details.find(',');
        std::size_t lastComma = details.rfind(',');
        if (firstComma == std::string::npos || firstComma == lastComma) {
            std::cout << "Invalid EMAIL command: " << details <<
std::endl;
            return;
        }
        std::string sender = trim(details.substr(0, firstComma));
        std::string subject =
            trim(details.substr(firstComma + 1, lastComma - firstComma -
1));
        std::string date = trim(details.substr(lastComma + 1));

        arrivalCounter++;
        queue.insert(Email(sender, subject, date, arrivalCounter));
    }

    // NEXT command: displays the highest priority email, leaving it in
the
    // queue so a second NEXT shows the same email.
    void showNext() const {
        if (queue.isEmpty()) {
            std::cout << "No emails to read." << std::endl;
            return;
        }
        std::cout << "Next email:" << std::endl;
        queue.peek().display();
    }
```

```
    // READ command: removes the highest priority email without displaying
it.
    void markRead() {
        if (!queue.removeMax()) {
            std::cout << "No emails to read." << std::endl;
        }
    }

    // COUNT command: displays the number of unread emails.
    void showCount() const {
        std::cout << "There are " << queue.size() << " emails to read."
                  << std::endl;
    }

    // Removes spaces, tabs and line-ending characters from both ends of
text.
    // Input: any string. Output: the trimmed string.
    static std::string trim(const std::string& text) {
        const std::string whitespace = " \t\r\n";
        std::size_t start = text.find_first_not_of(whitespace);
        if (start == std::string::npos) {
            return "";
        }
        std::size_t end = text.find_last_not_of(whitespace);
        return text.substr(start, end - start + 1);
    }
};

/*
 * main
 * Inputs: optional command-line argument holding the test file name.
 * Output: 0 on success, 1 if the test file could not be opened.
 */
int main(int argc, char* argv[]) {
    std::string fileName;
    if (argc > 1) {
        fileName = argv[1];
    } else {
        std::cout << "Enter the test file name: ";
```

```
        std::getline(std::cin, fileName);
    }

    Inbox inbox;
    if (!inbox.processFile(fileName)) {
        std::cout << "Error: could not open file " << fileName <<
std::endl;
        return 1;
    }
    return 0;
}
```